

ACCENTURE

Technical Test – Detailed Last-Minute Preparation Guide

High-yield preparation based primarily on the uploaded SRMIST Accenture Preparation Guide for the 2027 Batch

What this book contains: documented test structure, priority topics, pseudocode, SQL, OOP, OS, networking/security/cloud, DevOps, MS Office, DSA, HTML/CSS/JS, cognitive-game strategy, behavioural strategy, source examples, original practice questions, coding templates, exam tactics and final checklists.

Area	Pattern stated in uploaded guide
Online written test	125 questions / 175 minutes
Behavioural	54 questions / 20 minutes
Cognitive	Guide describes game-based reasoning; 3–4 questions / 20 minutes in its stated pattern
Technical	45 MCQs / 45 minutes
Coding	3 questions / 60 minutes
Coding categories	DSA/Basic Programming + SQL + Web (HTML/CSS/JS)

Source-label rule: This is a preparation guide, not an official archive of previous-year papers. Items shown in the uploaded guide are labelled as *guide examples*; new questions here are original practice questions and are not claimed to be actual PYQs.

Source locations: test structure pages 2–3; behavioural pages 4–5; cognitive pages 6–10; technical page 11; coding page 12; topic summary and examples pages 22–27.

1. EXAM BLUEPRINT & PRIORITY

The uploaded guide says the online written test has 125 questions in 175 minutes, covering behavioural/cognitive, technical and coding components. It lists 45 technical questions in 45 minutes and three coding questions in 60 minutes. [Source: uploaded guide, pages 2–3, 11–12]

Priority	Topic	Tonight's focus
★★★★★	Pseudocode	Loops, conditions, bitwise, tracing, output questions
★★★★★	SQL	JOIN, WHERE, GROUP BY, HAVING, aggregates, subqueries
★★★★★	OOP	Four pillars, overload/override, constructor, access modifiers
★★★★★	DSA	Arrays, strings, hash map/set, two pointers, complexity
★★★★■	HTML/CSS/JS	DOM, events, counter, arrays
★★★★■	OS	Processes, threads, scheduling, deadlock, memory
★★★★■	Networking/Security/Cloud	OSI, TCP/UDP, ports, authentication, cloud models
★★★██	DevOps	CI/CD, Docker vs VM
★★★██	MS Office	Word/Excel/PowerPoint basics
★★★██	Behavioural/Cognitive	Consistency + game strategy

Best order tonight: Pseudocode → SQL → OOP → DSA → OS/Networking → Web → remaining topics → final revision.

2. PSEUDOCODE – CORE CONCEPTS

Pseudocode is explicitly listed in the technical assessment topics, and the guide includes output-tracing examples on pages 20–21.

Operators

Operator	Meaning	Example
+	Addition	7+3=10
-	Subtraction	7-3=4
*	Multiplication	7*3=21
/	Integer division in integer context	17/5=3
%	Remainder	17%5=2
&& / AND	Logical AND	T AND F = F
/ OR	Logical OR	T OR F = T
!	Logical NOT	NOT T = F
&	Bitwise AND	6&5=4
	Bitwise OR	6 5=7
^	Bitwise XOR	6^5=3

Bitwise truth table

A	B	A&B	A B	A^B
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

6 = 110 and 5 = 101 → 6&5 = 100 = 4; 6^5 = 011 = 3.

Conditions

```
a = 10
b = 20
if(a > b)
  print a
else
  print b
```

Nested-if rule: evaluate the outer condition first. An inner block is irrelevant if its parent condition is false.

Loops

```
for(i = 0; i < n; i++)
  print i

i = 0
while(i < n)
  print i
  i = i + 1

i = 10
do
  print i
  i = i + 1
while(i < 5)
```

Do-while: executes at least once, even if the condition is initially false.

3. PSEUDOCODE – TRACING METHOD & TRAPS

Use this five-step method

- 1 Write all initial variable values.
- 2 Evaluate one statement at a time.
- 3 Write TRUE/FALSE for every condition.
- 4 For loops, record each iteration and updated variables.
- 5 Only then evaluate the final print statement.

High-frequency traps

- $17/5 \rightarrow 3$ in integer arithmetic; $17\%5 \rightarrow 2$.
- `i++` uses the old value first; `++i` increments before use.
- `&` and `^` are bitwise, not logical operators.
- Do not confuse assignment (`=`) with equality comparison (`==`) in languages where both exist.
- Do not skip a reassignment just because the variable was already initialized.
- For nested branches, follow the exact path that executes.

Guide example – DO-WHILE (page 20)

The guide shows `x=15`, `y=12`, then `y=x-1`, prints `x`, updates `x=y+(x-2)`, and repeats while `x<40`. The displayed answer choices include 15 27 39.

```
Initial: x=15, y=12
y = x - 1 -> 14

print x -> 15
x = 14 + (15 - 2) -> 27

print x -> 27
x = 14 + (27 - 2) -> 39

print x -> 39
x = 14 + (39 - 2) -> 51

51 < 40 is false -> stop
```

Guide-example answer: 15 27 39.

Guide example – nested condition + bitwise (page 21)

The guide shows `a=5`, `b=2`, `c=7` with nested conditions, bitwise operations and a final print of `a+b+c`. The reusable skill is branch-by-branch tracing rather than memorizing a particular answer.

4. PSEUDOCODE – 25 RAPID PRACTICE QUESTIONS

Q1. $7 \% 3 = ?$

Answer: 1

Q2. $6 \& 5 = ?$

Answer: 4

Q3. $6 \wedge 5 = ?$

Answer: 3

Q4. $x=5; y=x++;$ print $x+y$.

Answer: 11

Q5. $x=5; y=++x;$ print $x+y$.

Answer: 12

Q6. $\text{sum}=0;$ for $i=1..4:$ $\text{sum}=\text{sum}+i$. Final sum?

Answer: 10

Q7. $i=10;$ do print $i;$ while($i<5$). Print executions?

Answer: 1

Q8. $17 / 5$ with integer operands?

Answer: 3

Q9. $17 \% 5?$

Answer: 2

Q10. $a=10, b=20;$ if($a>b$) print a else print b .

Answer: 20

Q11. $x=2;$ repeat 3 times $x=x*2$. Final x ?

Answer: 16

Q12. $n=5;$ while($n>2$) $n=n-1$. Final n ?

Answer: 2

Q13. Maximum of $[4, 9, 2]$.

Answer: 9

Q14. Sum of even values in $[1, 2, 3, 4]$.

Answer: 6

Q15. $10 \& 7 = ?$

Answer: 2

Q16. $10 | 7 = ?$

Answer: 15

Q17. $10 \wedge 7 = ?$

Answer: 13

Q18. $a=3, b=4, c=a+b, a=c-b$. Final a ?

Answer: 3

Q19. for $i=0; i<5; i++:$ print i . Number of prints?

Answer: 5

Q20. If outer if is false, is its nested if executed?

Answer: No

Q21. $12 \% 4 = ?$

Answer: 0

Q22. $9 / 2$ in integer arithmetic?

Answer: 4

Q23. $5 \wedge 5 = ?$

Answer: 0

Q24. $8 \& 3 = ?$

Answer: 0

Q25. $8 | 3 = ?$

Answer: 11

Speed target: simple traces should take roughly 20–40 seconds. Mark long traces and return later.

5. SQL – CORE REVISION

SQL appears in the technical topic list and the coding round has a dedicated SQL query problem.

SELECT / WHERE / DISTINCT / ORDER BY

```
SELECT * FROM employees;
```

```
SELECT name, salary  
FROM employees  
WHERE salary > 50000;
```

```
SELECT DISTINCT department  
FROM employees;
```

```
SELECT name, salary  
FROM employees  
ORDER BY salary DESC;
```

Filtering

Construct	Use	Example
=	Equal	salary = 50000
> / <	Greater / less	salary > 50000
>= / <=	Boundary comparison	salary >= 50000
<>	Not equal	dept <> 'HR'
BETWEEN	Inclusive range	salary BETWEEN 30000 AND 60000
IN	One of listed values	dept IN ('IT','HR')
LIKE	Pattern match	name LIKE 'A%'

LIKE memory

- A% → starts with A
- %A → ends with A
- %A% → contains A

Aggregate functions

Function	Purpose
COUNT()	Count rows/values depending on expression
SUM()	Total
AVG()	Average
MIN()	Minimum
MAX()	Maximum

GROUP BY + HAVING

```
SELECT department, COUNT(*)  
FROM employees  
GROUP BY department  
HAVING COUNT(*) > 5;
```

Memory: WHERE filters rows; GROUP BY forms groups; HAVING filters groups.

6. SQL – JOINS, SUBQUERIES & CODING PATTERN

JOIN table

Join	What it keeps
INNER JOIN	Matching rows from both sides
LEFT JOIN	All left rows + matching right rows
RIGHT JOIN	All right rows + matching left rows
FULL OUTER JOIN	All rows from both sides where supported

```
SELECT e.name, d.dept_name
FROM Employee e
INNER JOIN Department d
ON e.dept_id = d.dept_id;
```

Second-highest salary

```
SELECT MAX(salary)
FROM employees
WHERE salary < (
    SELECT MAX(salary)
    FROM employees
);
```

Guide SQL example – page 26

The guide shows an e-commerce schema with Customers, Orders, Order_Items, Products, Order_Status and Countries. Its task asks for customer-level completed orders and total spending above a threshold, ordered by spending.

```
SELECT
    c.customer_id,
    c.customer_name,
    COUNT(DISTINCT o.order_id) AS total_orders,
    SUM(oi.quantity * oi.unit_price) AS total_amount
FROM Customers c
JOIN Orders o
    ON c.customer_id = o.customer_id
JOIN Order_Items oi
    ON o.order_id = oi.order_id
JOIN Order_Status os
    ON o.status_id = os.status_id
WHERE os.status_name = 'Completed'
GROUP BY c.customer_id, c.customer_name
HAVING SUM(oi.quantity * oi.unit_price) > 1000
ORDER BY total_amount DESC;
```

Reusable order: **JOIN** → **WHERE** → **GROUP BY** → **aggregate** → **HAVING** → **ORDER BY**.

7. SQL – 25 PRACTICE QUESTIONS

Q1. All employees with salary > 50,000.

Answer: `SELECT * FROM employees WHERE salary > 50000;`

Q2. Unique departments.

Answer: `SELECT DISTINCT department FROM employees;`

Q3. Highest salary first.

Answer: `SELECT * FROM employees ORDER BY salary DESC;`

Q4. Count employees.

Answer: `SELECT COUNT(*) FROM employees;`

Q5. Average salary.

Answer: `SELECT AVG(salary) FROM employees;`

Q6. Maximum salary.

Answer: `SELECT MAX(salary) FROM employees;`

Q7. Departments with >5 employees.

Answer: `SELECT department, COUNT(*) FROM employees GROUP BY department HAVING COUNT(*) > 5;`

Q8. Names starting with A.

Answer: `SELECT name FROM employees WHERE name LIKE 'A%';`

Q9. Salary 30k–60k.

Answer: `SELECT * FROM employees WHERE salary BETWEEN 30000 AND 60000;`

Q10. Employees in IT or HR.

Answer: `SELECT * FROM employees WHERE department IN ('IT','HR');`

Q11. Total salary per department.

Answer: `SELECT department, SUM(salary) FROM employees GROUP BY department;`

Q12. Average salary per department.

Answer: `SELECT department, AVG(salary) FROM employees GROUP BY department;`

Q13. Second-highest salary.

Answer: `MAX(salary) below the overall MAX(salary).`

Q14. Only matching employee/department rows.

Answer: `INNER JOIN.`

Q15. All employees even without a matching department.

Answer: `LEFT JOIN.`

Q16. Filter completed orders before aggregation.

Answer: `WHERE status='Completed'.`

Q17. Filter groups with spending >1000.

Answer: `HAVING SUM(...) > 1000.`

Q18. Sort spending high to low.

Answer: `ORDER BY total_amount DESC.`

Q19. Count unique orders.

Answer: `COUNT(DISTINCT order_id).`

Q20. Line-item amount.

Answer: `quantity * unit_price.`

Q21. What clause forms groups?

Answer: GROUP BY.

Q22. What clause filters groups?

Answer: HAVING.

Q23. What keyword removes duplicates?

Answer: DISTINCT.

Q24. What connects related tables?

Answer: JOIN ... ON ...

Q25. What wildcard means any sequence of characters in LIKE?

Answer: %

8. OOP – FOUR PILLARS + JAVA MCQ TRAPS

Pillar	Meaning	Typical clue
Encapsulation	Bundle data/methods and control direct access	private field + methods
Inheritance	Child derives from parent	extends
Polymorphism	Same interface/name can have multiple forms	overload/override
Abstraction	Hide implementation; expose essentials	interface/abstract class

Overloading vs overriding

Overloading	Overriding
Same name, different parameter list	Child redefines inherited method
Compile-time polymorphism	Runtime polymorphism
Usually within the same class	Requires inheritance

Constructor

```
class Student {  
    Student() {  
        System.out.println("Created");  
    }  
}  
Student s = new Student();
```

- Same name as class.
- No return type.
- Called during object creation.
- Used to initialize object state.

Access modifiers

private	Defining class
default/package-private	Same package in Java
protected	Package access plus inherited-class access under Java rules
public	Broadest accessibility

final / static

- final variable → cannot be reassigned.
- final method → cannot be overridden.
- final class → cannot be extended.
- static member → belongs to class rather than a particular object.

9. OOP – RAPID MCQs

Q1. Which pillar hides implementation details?

A) Inheritance B) Abstraction C) Compilation D) Sorting

Answer: B

Q2. Same method name, different parameters?

A) Overriding B) Overloading C) Inheritance D) Encapsulation

Answer: B

Q3. Java class inheritance keyword?

A) implements B) extends C) inherits D) super

Answer: B

Q4. Java interface implementation keyword?

A) extends B) implements C) interface D) instanceof

Answer: B

Q5. Data hiding is closely associated with?

A) Encapsulation B) Recursion C) Sorting D) Compilation

Answer: A

Q6. A final method can be overridden?

A) Yes B) No

Answer: B

Q7. A final class can be extended?

A) Yes B) No

Answer: B

Q8. Overriding requires?

A) Inheritance relationship B) Different parameters only C) No parent class D) SQL

Answer: A

Q9. Constructor has a return type?

A) Yes B) No

Answer: B

Q10. A class can implement multiple interfaces in Java?

A) Yes B) No

Answer: A

10. OPERATING SYSTEM

OS is explicitly listed in the technical topics.

Process vs Thread

Process	Thread
Separate process execution context/address space	Execution unit within a process
Higher overhead generally	Lower overhead generally
IPC may be needed for communication	Threads share process resources
More isolation	Less isolation inside same process

Process states

```
New → Ready → Running → Waiting/Blocked → Ready  
██████████████████████→ Terminated
```

CPU scheduling

FCFS	First Come First Serve
SJF	Shortest Job First
Round Robin	Time quantum in rotation
Priority	Selection based on assigned priority

Deadlock – four necessary conditions

- 1 Mutual Exclusion
- 2 Hold and Wait
- 3 No Preemption
- 4 Circular Wait

Mnemonic: **M-H-N-C.**

Memory

Register	Very fast CPU-access storage
Cache	Fast memory close to CPU
RAM	Volatile main memory
ROM	Non-volatile memory traditionally used for firmware/read-mostly data
SSD/HDD	Persistent secondary storage

11. NETWORKING & SECURITY

OSI model

7	Application	User-facing services
6	Presentation	Representation/translation/encryption concepts
5	Session	Session management
4	Transport	TCP/UDP
3	Network	IP/routing
2	Data Link	Frames/MAC
1	Physical	Bits/signals/media

Mnemonic: **Please Do Not Throw Sausage Pizza Away.**

TCP vs UDP

TCP	UDP
Connection-oriented	Connectionless
Reliable and ordered delivery semantics	No built-in delivery/order guarantee
Higher overhead	Lower overhead
Common where reliability matters	Common where low latency/low overhead matters

IP vs MAC

- IP = logical network addressing/routing.
- MAC = link-layer interface address on local network.

Common ports

HTTP	80
HTTPS	443
FTP	21
SSH	22
DNS	53
SMTP	25

Security vocabulary

Authentication	Who are you?
Authorization	What may you access/do?
Encryption	Plaintext → ciphertext using cryptographic mechanism/key
Firewall	Controls/filters network traffic
VPN	Secure/encrypted tunnel over an underlying network

Wi-Fi: remember WEP → WPA → WPA2 → WPA3 as a basic security-generation progression. SSID = Wi-Fi network name.

12. CLOUD & DEVOPS

Cloud service models

IaaS	Infrastructure such as VMs, storage, networking	Infrastructure
PaaS	Managed application platform/runtime	Platform
SaaS	Ready-to-use software delivered as a service	Software

Cloud deployment models

- Public cloud
- Private cloud
- Hybrid cloud = combination of public/private environments

DevOps

Code → Build → Test → Release → Deploy → Monitor

- CI = Continuous Integration: frequent integration with automated build/test practices.
- CD = Continuous Delivery/Deployment: automate release/deployment pipeline steps.
- Docker = containerization; containers share a host kernel in the standard Linux model.
- VM = virtual machine with virtualized hardware and generally a guest OS.

Rapid MCQs

Q1. Ready-to-use cloud software? **SaaS.**

Q2. Virtual machines/storage/networking as a service? **IaaS.**

Q3. Time-limited rotation in CPU scheduling? **Round Robin uses a time quantum.**

13. HTML / CSS / JAVASCRIPT

The coding round includes a Web Based Question (HTML/CSS/JS). The guide's page 27 shows a counter task using state, boundary conditions and a DOM status message.

HTML

```
<h1>Title</h1>
<p id="msg">Hello</p>
<button id="btn">Click</button>
<input id="name">
```

DOM

```
document.getElementById("msg")
document.querySelector("#msg")
document.querySelector(".card")

document.getElementById("msg").textContent = "Updated";
document.getElementById("name").value;
```

Events

```
button.addEventListener("click", function () {
  console.log("clicked");
});
```

Common JS array methods

push	Add to end
pop	Remove from end
shift	Remove from beginning
unshift	Add to beginning
slice	Return portion without in-place mutation
splice	Add/remove in place
map	Transform each element
filter	Keep elements matching condition
reduce	Accumulate to one result

map vs filter

```
[1,2,3].map(x => x*2)      // [2,4,6]
[1,2,3].filter(x => x>1)    // [2,3]
[1,2,3].reduce((s,x)=>s+x,0) // 6
```

CSS Flexbox

```
.container {
  display: flex;
  justify-content: center;
  align-items: center;
}
```

14. WEB CODING – GUIDE EXAMPLE & TEMPLATE

Guide example: Interactive Counter with Limit. The page-27 task asks for increment up to 10, decrement down to 0, and a status paragraph showing a maximum/minimum message at the boundaries.

```
let count = 0;

function updateStatus() {
  const status = document.getElementById("status");

  if (count === 10) {
    status.textContent = "Maximum limit reached";
  } else if (count === 0) {
    status.textContent = "Minimum limit reached";
  } else {
    status.textContent = "";
  }
}

function increment() {
  if (count < 10) {
    count++;
  }
  updateStatus();
}

function decrement() {
  if (count > 0) {
    count--;
  }
  updateStatus();
}
```

JS traps

- Array indexes start at 0.
- `textContent` changes displayed text; `input.value` reads/writes a form value.
- `querySelector` uses CSS-selector syntax.
- `map` transforms; `filter` selects; `reduce` accumulates.
- Use strict equality (`===`) when you want no type coercion.

15. DSA / BASIC PROGRAMMING

The guide specifies DSA/basic programming as the first coding question.

Core patterns

Pattern	Typical use
Single pass	sum/max/min/count
HashSet	duplicates/membership
HashMap	frequency/key-value lookup
Two pointers	sorted arrays/pairs
Sliding window	contiguous subarray/substring constraints
Stack	balanced brackets/LIFO processing
Queue	FIFO/BFS-like processing
Sorting	order data before greedy/two-pointer logic

Complexity

O(1)	Direct access/simple operation
O(log n)	Binary search
O(n)	Single pass
O(n log n)	Common efficient sorting
O(n ²)	Nested loops over n
O(2 ⁿ)	Some exhaustive recursion
O(n!)	Some permutation generation

Python templates

```
from collections import Counter

# frequency
freq = Counter(arr)

# duplicate detection
seen = set()
for x in arr:
    if x in seen:
        print("Duplicate")
    seen.add(x)

# max
mx = arr[0]
for x in arr[1:]:
    mx = max(mx, x)
```

Two Sum

```
def two_sum(nums, target):
    seen = {}
    for i, x in enumerate(nums):
        need = target - x
        if need in seen:
            return [seen[need], i]
        seen[x] = i
    return []
```

16. DSA – GUIDE EXAMPLES & PRACTICE

Guide example – index-based array transformation (page 24)

The guide asks you to transform each element using its 0-based index, then apply an additional divisibility-by-11 condition and return the total sum. The reusable pattern is: iterate → retain original value → compute index-dependent modification → test condition → accumulate.

Guide example – Equivalent Sum (page 25)

For $X=112$, prefixes are 1, 11, 112, so $\text{EqSum}(112)=1+11+112=124$.

```
def eq_sum(n):
    s = str(n)
    total = 0
    for i in range(1, len(s)+1):
        total += int(s[:i])
    return total
```

Must-practice problems

- maximum/minimum
- second largest
- reverse array/string
- palindrome
- frequency count
- remove duplicates
- two sum
- missing number
- move zeroes
- longest consecutive sequence
- balanced parentheses
- merge sorted arrays

Rapid DSA questions

Q1. Sum [2,4,6,8].

Answer: 20

Q2. Maximum [3,9,2,7].

Answer: 9

Q3. Is 'level' a palindrome?

Answer: Yes

Q4. Frequency of 2 in [1,2,2,3,2].

Answer: 3

Q5. Pair summing to 13 in [1,5,8,10]?

Answer: Yes: 5+8

Q6. Complexity of one loop over n.

Answer: $O(n)$

Q7. Complexity of nested n-by-n loops.

Answer: $O(n^2)$

Q8. Best common structure for unique membership checks?

Answer: Set/HashSet

Q9. Key→value lookup?

Answer: Dictionary/HashMap

Q10. Stack order?

Answer: LIFO

Q11. Queue order?

Answer: FIFO

Q12. Binary search requires ordered/sorted data?

Answer: Yes, in its standard form

Q13. Reverse [1,2,3].

Answer: [3,2,1]

Q14. Missing from [1,2,4,5] in 1..5?

Answer: 3

Q15. EqSum(123)?

Answer: 136

17. COMMON APPLICATIONS & MS OFFICE

The guide explicitly lists Common Applications and MS Office.

Word

- Formatting/styles
- Headers/footers
- Page breaks
- Find & Replace
- Mail Merge
- Spelling/grammar tools

Excel

Function	Use
SUM()	Add values
AVERAGE()	Arithmetic mean
COUNT()	Count numeric cells
MAX()	Maximum
MIN()	Minimum
IF()	Conditional result

=SUM(A1:A10)

=AVERAGE(B1:B10)

=IF(C2>=50, "Pass", "Fail")

PowerPoint

- Themes
- Layouts
- Transitions
- Animations
- Slide Master

18. BEHAVIOURAL ASSESSMENT

The guide states 54 behavioural questions in 20 minutes and highlights thought process, leadership, collaboration, problem solving, continuous learning, business ethics and emotional resilience.

The guide's example directions

Statement pattern	Guide example direction	Trait
Calm under tight deadlines	Highly Agree	Pressure handling
Prefer people who think exactly like me	Disagree	Teamwork/adaptability
Use constructive feedback	Highly Agree	Growth mindset
Hide mistakes unless noticed	Highly Disagree	Accountability/integrity
Comfortable learning unfamiliar technology	Highly Agree	Adaptability/learning

Practical approach

- Be internally consistent.
- For mistakes, favor accountability and corrective action rather than concealment.
- For teamwork, recognize differing opinions can be useful.
- For learning, show genuine openness to unfamiliar technologies.
- Do not blindly choose extreme answers; answer truthfully.

19. COGNITIVE / GAME-BASED ASSESSMENT

The guide describes Memory Maze/Lock and Door, Fast Math Challenges and Grid/Path Finding. It says movement in the maze is up/down/left/right only, walls remain fixed, you can retrace, and you must collect the key before the door.

Memory Maze strategy

- 1 Mark discovered walls mentally.
- 2 Remember safe paths rather than repeating random attempts.
- 3 Plan key → door.
- 4 Use failed attempts as information because walls do not move.

Fast Math

Expression	Value
4-5	-1
11-13	-2
0+3	3
12×8	96
140÷2	70
15×6	90

Path Finder

Look for how adjacent tiles/path segments connect to the destination. Avoid spending too long on a single arrangement; use the visible route structure systematically.

20. ORIGINAL TECHNICAL MOCK – 35 MCQs

These are original practice questions modelled on the uploaded guide's topic list. They are not official PYQs.

Q1. $6 \& 5 = ?$

A)1 B)4 C)3 D)7

Answer: B

Q2. $17 \% 5 = ?$

A)1 B)2 C)3 D)4

Answer: B

Q3. Which loop executes at least once?

A)for B)while C)do-while D)none

Answer: C

Q4. Compile-time polymorphism?

A)Overriding B)Overloading C)Inheritance D)Abstraction

Answer: B

Q5. Java class inheritance keyword?

A)implements B)extends C)inherits D)super

Answer: B

Q6. WHERE filters primarily:

A)groups B)rows C)tables only D)indexes

Answer: B

Q7. HAVING commonly follows:

A)GROUP BY B)DROP C)INSERT D)CREATE

Answer: A

Q8. Unique results keyword:

A)UNIQUE B)DISTINCT C)ONLY D)DIFFERENT

Answer: B

Q9. HTTP port:

A)22 B)53 C)80 D)443

Answer: C

Q10. HTTPS port:

A)25 B)80 C)443 D)21

Answer: C

Q11. DNS port:

A)53 B)22 C)443 D)110

Answer: A

Q12. Transport-layer protocol:

A)IP B)TCP C)MAC D)Ethernet

Answer: B

Q13. Logical network addressing:

A)IP B)MAC C)SSID D)WPA

Answer: A

Q14. Volatile memory:

A)ROM B)RAM C)SSD D)HDD

Answer: B

Q15. Fast memory near CPU:

A)Cache B)HDD C)DVD D)USB

Answer: A

Q16. Deadlock cyclic condition:

A)Mutual exclusion B)Circular wait C)Preemption D)Priority

Answer: B

Q17. Round Robin uses:

A)time quantum B)only priority C)only SJF D)no queue

Answer: A

Q18. IaaS means:

A)Infrastructure as a Service B)Internet as a Service C)Interface as a Service D)Identity as a Service

Answer: A

Q19. Ready-to-use software service:

A)IaaS B)PaaS C)SaaS D)FaaS

Answer: C

Q20. Traffic filtering control:

A)Firewall B)Compiler C)Cache D)Spreadsheet

Answer: A

Q21. Authentication asks:

A)What access? B)Who are you? C)Which route? D)Which port?

Answer: B

Q22. DOM by ID:

A)getById B)getElementById C)selectId D)queryId

Answer: B

Q23. Transform each array element:

A)filter B)map C)push D)pop

Answer: B

Q24. Keep elements meeting condition:

A)map B)filter C)reduce D)shift

Answer: B

Q25. Key→value lookup:

A)HashMap B)Stack C)Queue D)Tree only

Answer: A

Q26. Membership/duplicates:

A)HashSet B)Queue C)Array only D)Compiler

Answer: A

Q27. Binary search:

A) $O(n^2)$ B) $O(n)$ C) $O(\log n)$ D) $O(n!)$

Answer: C

Q28. Stack:

A)FIFO B)LIFO C)priority D)random

Answer: B

Q29. Queue:

A)FIFO B)LIFO C)tree-order D)hash-order

Answer: A

Q30. Excel sum function:

A)ADD B)SUM C)TOTAL D)PLUS

Answer: B

Q31. Wi-Fi network name:

A)SSID B)MAC C)DNS D)WPA

Answer: A

Q32. Docker primarily provides:

A)containerization B)word processing C)SQL joins D)CPU scheduling

Answer: A

Q33. PaaS provides primarily:

A)managed platform B)raw hardware only C)finished end-user app only D)Wi-Fi

Answer: A

Q34. Which is a necessary deadlock condition?

A)Circular wait B)Compilation C)Sorting D)Caching

Answer: A

Q35. CSS property for flex layout:

A)display:flex B)font:flex C)layout:flexbox-only D)position:flex

Answer: A

21. CODING ROUND – 3-PROBLEM MOCK

The uploaded guide's coding section has three categories: DSA/basic programming, SQL and Web (HTML/CSS/JS), with 60 minutes total.

Problem 1 – DSA

Task: Given an integer array, return the sum of all even values that occur exactly once.

Example: [2,2,4,5,6] → 10.

```
from collections import Counter

def solve(arr):
    freq = Counter(arr)
    total = 0
    for x in arr:
        if x % 2 == 0 and freq[x] == 1:
            total += x
    return total
```

Problem 2 – SQL

Employees(emp_id,name,dept_id,salary), Departments(dept_id,dept_name). Return departments with average salary > 50,000, sorted descending by average.

```
SELECT d.dept_name, AVG(e.salary) AS avg_salary
FROM Employees e
JOIN Departments d
ON e.dept_id = d.dept_id
GROUP BY d.dept_id, d.dept_name
HAVING AVG(e.salary) > 50000
ORDER BY avg_salary DESC;
```

Problem 3 – Web

Build a counter from 0 to 10. Increment/decrement must respect limits and a status paragraph must show the appropriate boundary message.

```
let count = 0;

function inc() {
    if (count < 10) count++;
    update();
}

function dec() {
    if (count > 0) count--;
    update();
}

function update() {
    const s = document.getElementById("status");
    s.textContent =
        count === 10 ? "Maximum limit reached" :
        count === 0 ? "Minimum limit reached" : "";
}
```

22. EXAM-DAY TACTICS & CHECKLIST

Technical section: 45 questions in 45 minutes in the uploaded guide.

If you see...	Do this
Easy factual MCQ	Answer immediately.
Simple pseudocode	Trace quickly.
Long pseudocode	Mark and move.
Unknown theory	Eliminate clearly wrong options; don't burn several minutes.
Last 5 minutes	Review marked questions.

Coding section

Suggested time	Action
0–5 min	Read all three problems.
5–30 min	DSA/basic programming.
30–45 min	SQL.
45–58 min	Web.
58–60 min	Edge cases + submission.

Edge cases

- empty/small input
- single element
- duplicates
- all equal
- zero/boundary values
- negative values if permitted
- no match
- maximum/minimum

Pseudocode rule: **never guess an output if you can trace it.**

SQL rule: **identify keys and joins before writing aggregation.**

Coding rule: **correct basic solution first; optimize only if time remains.**

Exam-integrity reminder: use only resources and devices permitted by the secure exam environment and follow its instructions.

23. FINAL 10-MINUTE REVISION SHEET

Pseudocode

% = remainder
/ = integer division in integer context
i++ = use, then increment
++i = increment, then use
& = bitwise AND
| = bitwise OR
^ = bitwise XOR
&& = logical AND
|| = logical OR
! = logical NOT
do-while = body executes at least once

SQL

SELECT ... FROM ...
WHERE ... -- rows
GROUP BY ... -- groups
HAVING ... -- groups after aggregation
ORDER BY ... ASC/DESC
COUNT / SUM / AVG / MIN / MAX
JOIN ... ON ...
DISTINCT
LIKE 'A%' / '%A' / '%A%'
BETWEEN ... AND ...
IN (...)

OOP

Encapsulation → data hiding/bundling
Inheritance → child derives from parent
Polymorphism → many forms
Abstraction → hide implementation
Overloading → same name, different parameters
Overriding → child redefines inherited method
extends → class inheritance
implements → interface implementation

OS / Networking

Deadlock: Mutual Exclusion + Hold/Wait + No Preemption + Circular Wait

OSI:
Application
Presentation
Session
Transport
Network
Data Link
Physical

HTTP 80 | HTTPS 443 | FTP 21 | SSH 22 | DNS 53 | SMTP 25
TCP = connection-oriented/reliable
UDP = connectionless/lower overhead

Cloud / Web

IaaS = Infrastructure
PaaS = Platform
SaaS = Software

DOM:
getElementById()
querySelector()
textContent
value
addEventListener()

map = transform
filter = select
reduce = accumulate

24. FINAL NIGHT / MORNING PLAN

Time block	Action
45 min	Pseudocode: operators, loops, bitwise, 20–25 practice questions
45 min	SQL + OOP
35 min	DSA + complexity + common patterns
30 min	OS + networking + cloud/security
25 min	HTML/CSS/JS + counter
15 min	Cognitive/behavioural strategy
10 min	Final cheat sheet only

If only 20 minutes remain

Pseudocode → SQL → OOP → DSA complexity → OSI/ports → JS DOM.

If only 10 minutes remain

Read the final revision sheet once, breathe, and stop cramming.

Morning checklist

- Laptop charged / power connected if permitted.
- Required secure exam setup ready.
- No unauthorized notes/devices/tabs.
- Read every pseudocode branch carefully.
- Use JOIN → WHERE → GROUP BY → HAVING → ORDER BY for aggregation problems.
- Test coding edge cases.
- Manage the clock; don't let one question consume the section.
- Keep behavioural responses consistent and truthful.

You are not trying to know every CS topic tonight. You are trying to recognize the patterns that this guide emphasizes and execute them accurately under time pressure.

Good luck for tomorrow morning.